```
css = (raw) ->
  hyphenate = (property) ->
    dashThenUpperAsLower = (match, pre, upper) ->
      "#{pre}-#{upper.toLowerCase()}"
    property.replace /([a-z])([A-Z])/g, dashThenUpperAsLower

  output = (for selector, rules of raw          #B
    rules = (for ruleName, ruleValue of rules
      "#{hyphenate ruleName}: #{ruleValue};"
    ).join '\n'
    """
    #{selector} {
      #{rules}
    }
    """
  ).join '\n'
```

CSS attributes use dashes, but only camel case is possible in CoffeeScript. This function converts camel case property names to hyphenated property names.

The final novel DSL you will look at is an SQL DSL in CoffeeScript.

SQL

With CoffeeScript's liberal syntax, it's tempting to attempt an SQL DSL that looks just like regular SQL by endlessly chaining functions:

```
SELECT '*' FROM 'users' WHERE 'name LIKE "%scruffy%"'
```

But SQL syntax is a little more complicated than you might think, and when you mix that with CoffeeScript's significant indentation, it can get ugly. The simplest way to implement an SQL DSL in CoffeeScript is to use the object literal style:

```
query
  SELECT: '*'
  FROM: 'users'
  WHERE: 'name LIKE "%scruffy%"'
```

Remember, property values can be evaluated at runtime. This can be powerful, but it might make your DSL too permissive:

```
query
  SELECT: '*'
  FROM: 'users'
  WHERE: "name LIKE '%#{session.user.name}%'"
```